

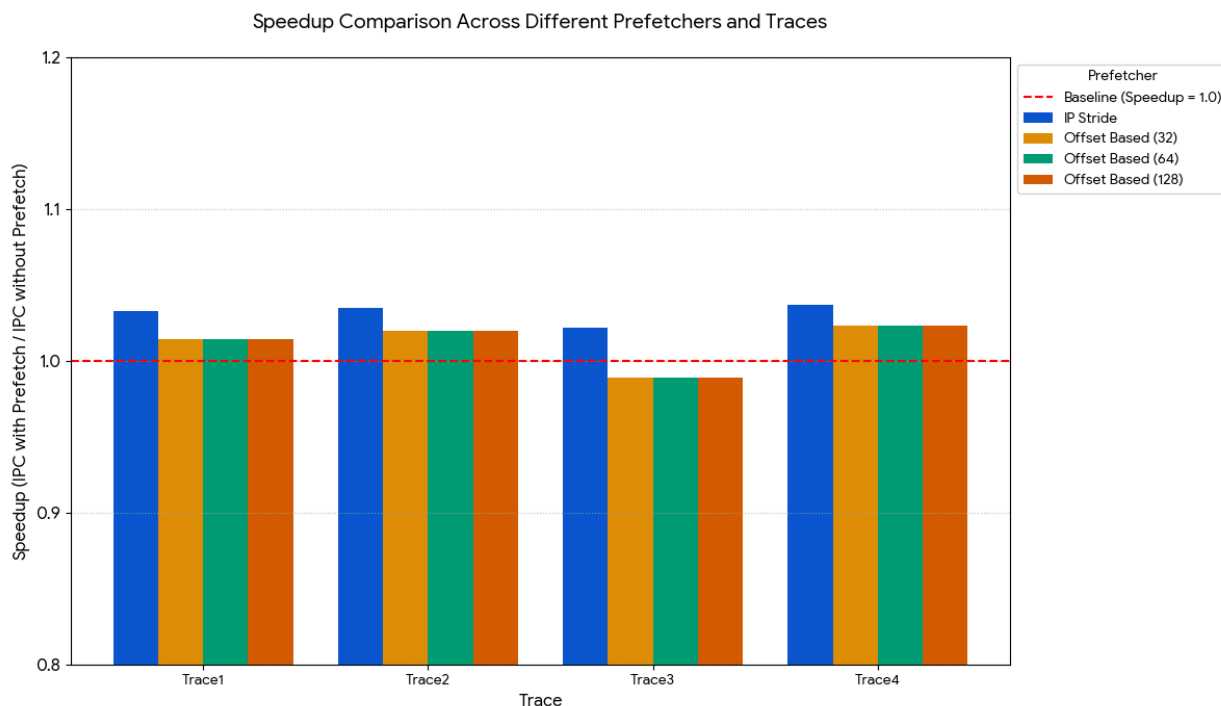
Task 1

IP Stride				
	Trace1	Trace2	Trace3	Trace4
IPC	0.54648	0.478241	0.852268	0.47923
IPC(no prefetch)	0.529026	0.462268	0.834309	0.462276
Speedup	1.032992707	1.034553549	1.021525598	1.03667506
L2 MPKI	40.3407	47.7267	22.1965	47.5704

Offset Based (Table Size 32)				
	Trace1	Trace2	Trace3	Trace4
IPC	0.536574	0.471457	0.825139	0.472948
IPC(no prefetch)	0.529026	0.462268	0.834309	0.462276
Speedup	1.01426773	1.019878079	0.9890088684	1.023085776
L2 MPKI	40.3454	47.7005	22.1962	47.5694

Offset Based (Table Size 64)				
	Trace1	Trace2	Trace3	Trace4
IPC	0.536574	0.471457	0.825139	0.472948
IPC(no prefetch)	0.529026	0.462268	0.834309	0.462276
Speedup	1.01426773	1.019878079	0.9890088684	1.023085776
L2 MPKI	40.3454	47.7005	22.1962	47.5694

Offset Based (Table Size 128)				
	Trace1	Trace2	Trace3	Trace4
IPC	0.53662	0.471457	0.825139	0.472948
IPC(no prefetch)	0.529026	0.462268	0.834309	0.462276
Speedup	1.014354682	1.019878079	0.9890088684	1.023085776
L2 MPKI	40.3438	47.7005	22.1962	47.5694



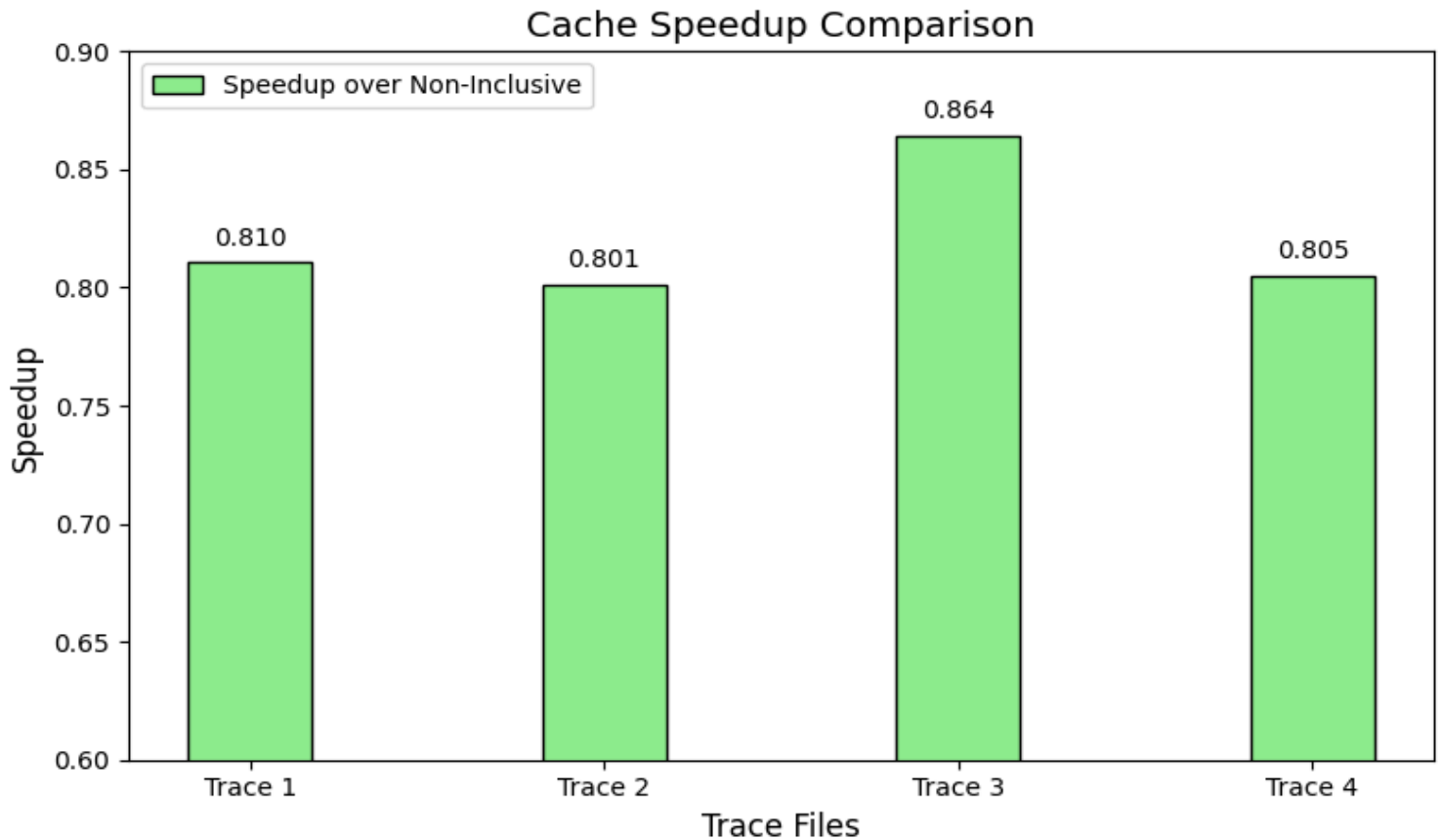
The prefetcher uses 32(can change to 64,128, etc.) trackers (set by TRACKER_COUNT) to store information about pages. Each tracker can remember up to 5 offsets (MAX_OFFSETS) within the page. Along with offsets, the tracker records how often each offset is used, the number of prefetches issued, and the number of prefetches that were actually useful. Trackers are replaced using a Least Recently Used (LRU) policy when needed.

When the CPU accesses memory, the code calculates the **page number** and the **offset** within that page. If the access hits a previously prefetched line, the prefetch is counted as **useful**. If no tracker exists for the page, the prefetcher **allocates a new tracker** by replacing the least recently used one. The current offset is updated in the tracker by either increasing its frequency or replacing the **least used offset**.

The code then decides which offsets to prefetch next. It uses the most frequently used offset to predict the next memory accesses. The prefetch degree, or the number of lines to prefetch, is based on accuracy: if many previous prefetches were useful, it prefetches more lines (up to 5). Prefetches are limited to the same page to avoid invalid memory access. Depending on MSHR occupancy, the prefetch is sent to the L2 cache or LLC.

Task 2 :

	IPC	MPKI-L1D	MPKI-L2	MPKI-LLC	Time	Speedup Over Non-inclusive (Using IPC)
Exclusive	0.429213	66.0504	96.9167	86.257	191	0.8105
Non-Inclusive	0.529507	66.0505	40.5813	40.1098	172	
Exclusive	0.370787	78.0223	114.36	101.361	212	0.80125
Non-Inclusive	0.46276	78.0219	47.742	47.3799	184	
Exclusive	0.725385	35.7545	52.5689	46.2936	107	0.86432
Non-Inclusive	0.839253	35.7547	22.0985	21.7126	99	
Exclusive	0.371905	77.7237	113.959	101.011	227	0.804876
Non-Inclusive	0.462065	77.7237	47.6566	47.1964	200	



Logic Used:

1. **Handle_fill:** After adding the block to the writeback queue, make the current block invalid and non-dirty, as a new block will be brought in its place. If the fill request is done for L1, then invalidating L2C and LLC entries if they exist. If the fill request is done for L2 also the fill level is L2, then invalidating L1 and LLC. For LLC, fill only when fill_level belongs to LLC otherwise, skip invalidation. If the request came from the upper layer cache, then invalidating the current entry in cache and returning the data to the upper layer. For every handle_fill request the MSHR removal will happen as usual.

2. **Handle_writeback:** In case of miss, if the current block is dirty, then add it to the writeback queue and make the block non-valid and non-dirty. After filling the cache, if the writeback request came from the upper layer, then invalidating the upper layer entries, as the block is already present in the current cache after doing fill_cache, resetting the fillvalues to 0. If from the LLC to the next level i.e., DRAM, a writeback request is added, then, after fulfilling it block is sent to the LLC.

3. **Handle_read:** In case of a hit, if the read request came from upper-level cache, then invalidate the current entry and return the requested block to upper-level cache. For miss, read request handled as usual.

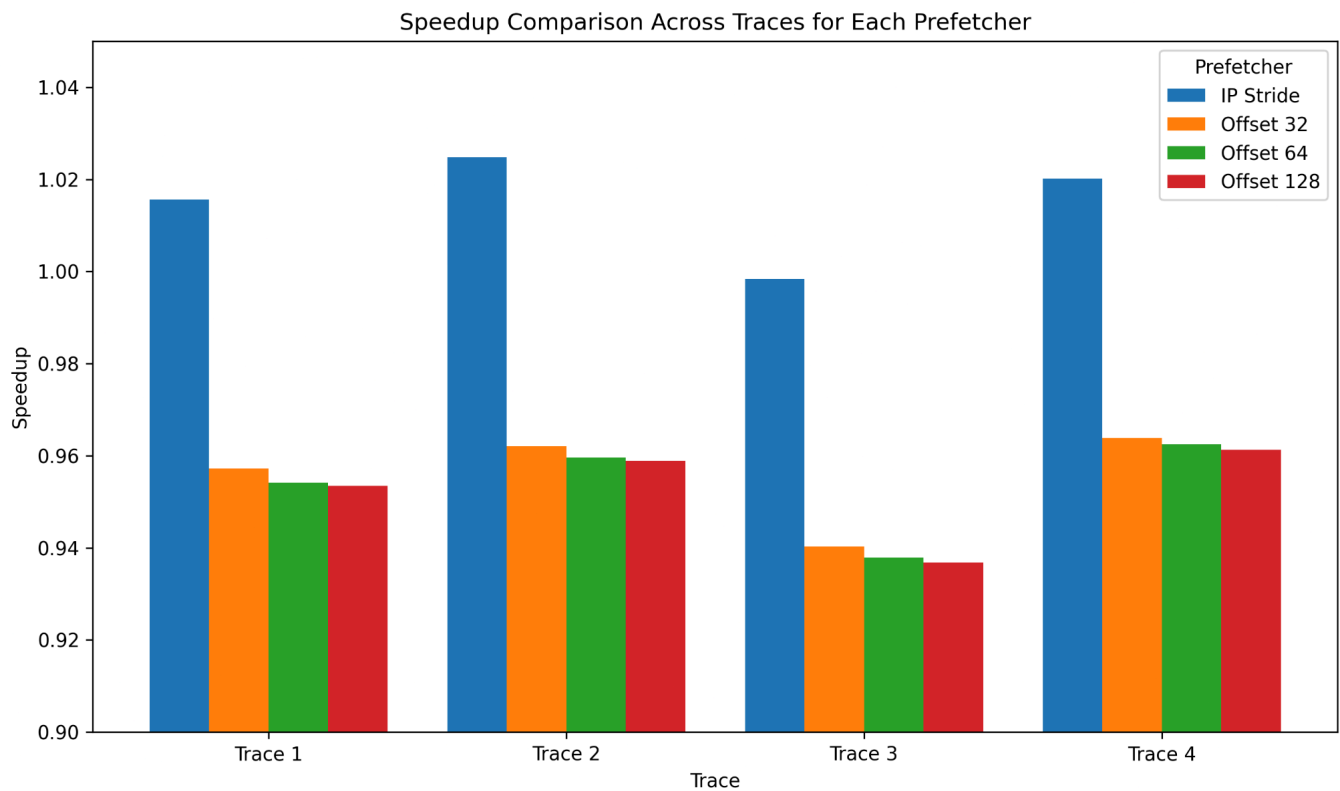
Task 3:

For exclusive baseline comparison

	Trace 1			
	IP Stride	Offset 32	Offset 64	Offset 128
IPC	0.435902	0.410836	0.409544	0.409223
IPC(no prefetch)	0.429213			
Speedup	1.01558	0.9572	0.95417	0.95343
L2 MPKI	96.936	98.9239	99.0293	99.0247

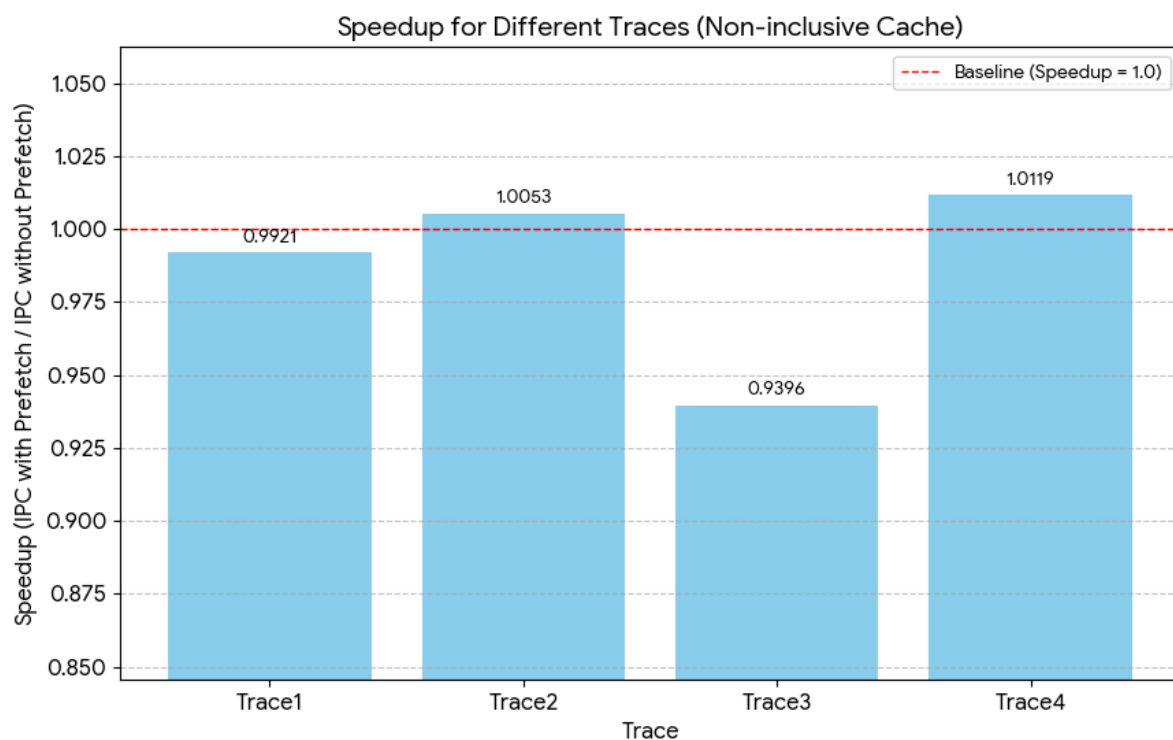
Trace 2			
			Offset 128
IP Stride	Offset 32	Offset 64	
0.378	0.356731	0.355824	0.355557
0.370787			
1.02484	0.96209	0.9596	0.95892
114.392	116.672	116.751	116.789

Trace 3				Trace 4				
	IP Stride	Offset 32	Offset 64	Offset 128	IP Stride	Offset 32	Offset 64	Offset 128
IPC	0.724238	0.682061	0.680333	0.679523	0.379415	0.358464	0.357942	0.357486
IPC(no prefetch)	0.725385				0.371905			
Speedup	0.9984	0.94027	0.93789	0.9367756	1.02019	0.96385	0.96245	0.96123
L2 MPKI	52.5727	53.8028	53.8215	53.7575	113.972	116.331	116.404	116.404



For Non-inclusive baseline comparison

Non-inclusive cache (without prefetching)				
	Trace1	Trace2	Trace3	Trace4
IPC	0.524865	0.464739	0.783945	0.467754
IPC(no prefetch)	0.529026	0.462268	0.834309	0.462276
Speedup	0.9921346021	1.005345384	0.9396338767	1.011850064
L2 MPKI	40.3447	47.6951	22.1967	47.5673



Logic Used:

For writebacks, it is marked as non-critical request and send to the lower level i.e. LLC. Prefetched lines for writebacks are sent to the LLC instead of L2.